

Tecnológico Nacional de México
Instituto Tecnológico de Mazatlán
Ingeniería en Sistemas Computacionales

Reporte de práctica

Buffer circular con interrupción externa

Arduino Uno R4 WiFi · Matriz LED integrada · Pulsador en interrupción (D2)

Dato	Detalle
Materia	Sistemas Programables
Práctica	Buffer circular con interrupción externa
Número de control	231000103
Placa	Arduino Uno R4 WiFi
Herramientas	Arduino IDE 2, Tinkercad Circuits
Fecha de pruebas	16 de septiembre de 2026
Fecha de entrega	17 de septiembre de 2026

Resumen

Se implementó un registro de eventos con un pulsador conectado a una interrupción externa (pin D2, modo FALLING) en un Arduino Uno R4 WiFi. Cada pulsación se guardó en un buffer circular de 32 posiciones, con su marca de tiempo en microsegundos y un filtro de antirrebote de 40 ms aplicado dentro de la propia rutina de interrupción. El loop() retiró los eventos del buffer, los contó y los mostró por el Monitor Serial, mientras la matriz LED integrada mantuvo una animación en movimiento sin usar delay() en ningún momento. Las pruebas registraron 30 eventos consecutivos sin pérdidas ni desorden.

I. Introducción

Un microcontrolador que espera un evento impredecible —como el paso de una pieza frente a un sensor— enfrenta un problema si, al mismo tiempo, debe seguir ejecutando otra tarea continua, como una animación. Revisar el sensor constantemente (sondeo) puede hacer que se pierda un evento si ocurre justo cuando el programa está ocupado en otra parte del código. Las interrupciones externas [1] resuelven esto: el hardware avisa al programa de inmediato, sin importar en qué instrucción se encuentre, y salta a una función dedicada. Esta práctica combina una interrupción externa con un buffer circular [5], una estructura de datos de tamaño fijo que permite que la interrupción anote eventos muy rápido y que el programa principal los procese después, a su propio ritmo, sin bloquear ni perder ninguno.

II. Objetivos

Objetivo general. Registrar las pulsaciones de un botón mediante una interrupción externa y almacenarlas temporalmente en un buffer circular, procesándolas en orden sin perder ninguna, mientras una animación en la matriz LED se mantiene en movimiento constante.

- Configurar una interrupción externa (FALLING) en el pin D2 para detectar cada pulsación sin necesidad de revisarlo constantemente en el loop().
- Implementar un buffer circular de 32 posiciones con índices de escritura y lectura independientes, distinguiendo el estado lleno del vacío.
- Aplicar un filtro de antirrebote (40 ms) dentro de la rutina de interrupción, en vez de dentro del loop().
- Mantener una animación no bloqueante en la matriz LED con millis() mientras se atienden los eventos de la interrupción.

III. Marco teórico

A. Interrupciones externas frente al sondeo

Leer un botón dentro del loop() (sondeo) funciona bien cuando el programa no tiene otra tarea urgente, pero si el Arduino está ocupado ejecutando otro bloque de código —como actualizar una animación— puede tardar en llegar de nuevo a esa línea y perderse un evento breve. Una interrupción externa [1] evita este problema: el pin se configura para que, ante un cambio de voltaje específico (en este caso, de HIGH a LOW, modo FALLING), el microcontrolador pause momentáneamente lo que está haciendo y ejecute de inmediato una función dedicada (la rutina de servicio de interrupción o ISR). La regla general es que esa función debe ser lo más corta posible, porque mientras se ejecuta, el resto del programa queda pausado.

B. Buffers circulares

Un buffer circular es un arreglo de tamaño fijo que se recorre en forma de círculo: al llegar a la última posición, el siguiente dato se escribe de nuevo en la primera. Se administra con dos índices independientes —uno de escritura y uno de lectura— que avanzan por separado: la interrupción escribe en el índice de escritura y lo avanza; el `loop()` lee desde el índice de lectura y lo avanza por su cuenta. El reto habitual de esta estructura es distinguir el buffer completamente lleno del completamente vacío, ya que en ambos casos los dos índices pueden apuntar a la misma posición; esta práctica lo resuelve con una variable adicional que cuenta los eventos pendientes de leer.

C. Antirrebote dentro de una interrupción

Un botón mecánico no cambia de HIGH a LOW de forma limpia: el contacto vibra varias veces en pocos milisegundos [2]. Con lectura en el `loop()` el rebote se filtra fácilmente, pero una interrupción reacciona a cada cambio de voltaje, por lo que un solo clic físico podría generar varias interrupciones seguidas. Por eso, dentro de la propia función de interrupción se agrega una comprobación mínima de tiempo transcurrido desde el último evento aceptado (en esta práctica, 40 ms) [4], adaptando la técnica de antirrebote para que viva dentro de la ISR en vez de dentro del `loop()`.

IV. Materiales y herramientas

Tabla I. Materiales y herramientas.

Elemento	Uso en la práctica
Arduino Uno R4 WiFi	Ejecuta la interrupción, administra el buffer circular y controla la animación
Pulsador de cuatro patas	Genera el aviso de una pulsación; simula el sensor de piezas
Protoboard	Permite armar el circuito sin soldar los componentes
2 cables de conexión	Conectan el pulsador entre el pin D2 y GND
Cable USB-C	Alimenta la placa, carga el programa y permite consultar el Monitor Serial
Arduino IDE 2	Programación de la placa
Tinkercad Circuits	Elaboración del diagrama de conexiones

V. Desarrollo

A. Circuito

El pulsador se conectó en la protoboard entre el pin D2 y GND del Arduino, usando la resistencia de pull-up interna (`INPUT_PULLUP`) para que el pin quede en HIGH por defecto y pase a LOW al presionarlo, sin necesidad de una resistencia externa. La matriz LED utilizada es la que ya está integrada en la placa Uno R4 WiFi. La Figura 1 muestra el diagrama elaborado en Tinkercad Circuits, y la Tabla II detalla las conexiones.

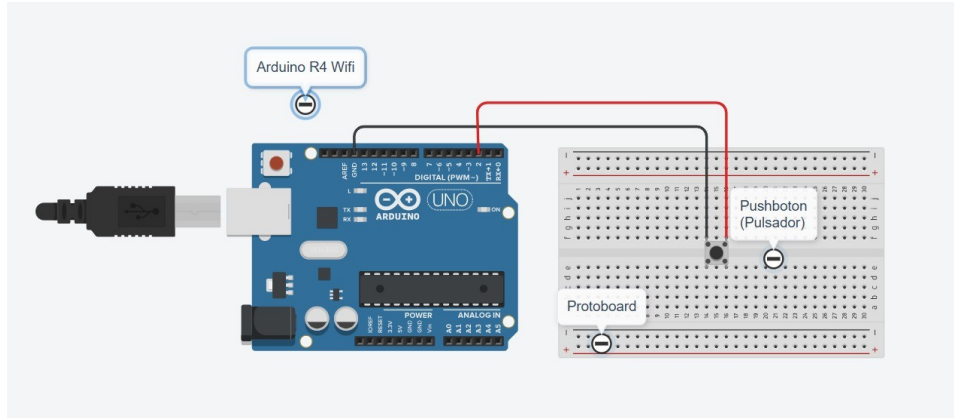


Figura 1. Diagrama de conexiones del pulsador (Tinkercad Circuits).

Tabla II. Conexiones del circuito.

Elemento	Conexión	Función
Pulsador	Dos contactos que se unen al pulsar: uno a D2 y otro a GND	Generar el aviso de una pulsación
Entrada D2	INPUT_PULLUP e interrupción FALLING	Detectar la transición de HIGH a LOW
Matriz LED	Integrada en el Uno R4 WiFi	Mostrar una animación en movimiento

B. Programa

La interrupción (ISR). La función registrarEvento() se ejecuta cada vez que el pin D2 cambia de HIGH a LOW. Consulta micros() [3], aplica el filtro de 40 ms y, si hay espacio libre, guarda el instante del evento en el buffer de 32 posiciones y avanza el índice de escritura. No imprime nada ni realiza cálculos adicionales, para mantenerse lo más breve posible.

Organización del buffer circular. El buffer tiene 32 posiciones. La Tabla III resume las variables que lo administran.

Tabla III. Organización del buffer circular.

Variable o condición	Función en el programa
indiceEscritura / indiceLectura	Indican dónde guardar y de dónde retirar el siguiente evento. Al llegar a 31 regresan a 0.
eventosPendientes	Distingue el estado vacío (0) del lleno (32), aunque los índices coincidan.
Buffer lleno	Se conservan los datos pendientes; los eventos nuevos se rechazan mientras no haya espacio libre.

Lectura en el loop(). obtenerEvento() retira un dato del buffer por cada vuelta del loop(), protegiendo brevemente las variables compartidas con la interrupción, e incrementa el contador que se imprime por el Monitor Serial.

Animación de la matriz LED. actualizarAnimacion() desplaza una línea de LEDs en la matriz integrada cada 100 ms usando millis(), sin usar delay() en ningún momento, para que la animación nunca se detenga aunque el pulsador se presione repetidamente.

VI. Resultados

El montaje se probó presionando el pulsador repetidamente mientras se observaba el Monitor Serial y la animación de la matriz LED; el video de la práctica documenta el funcionamiento completo. La Figura 2 muestra el armado físico usado en las pruebas.

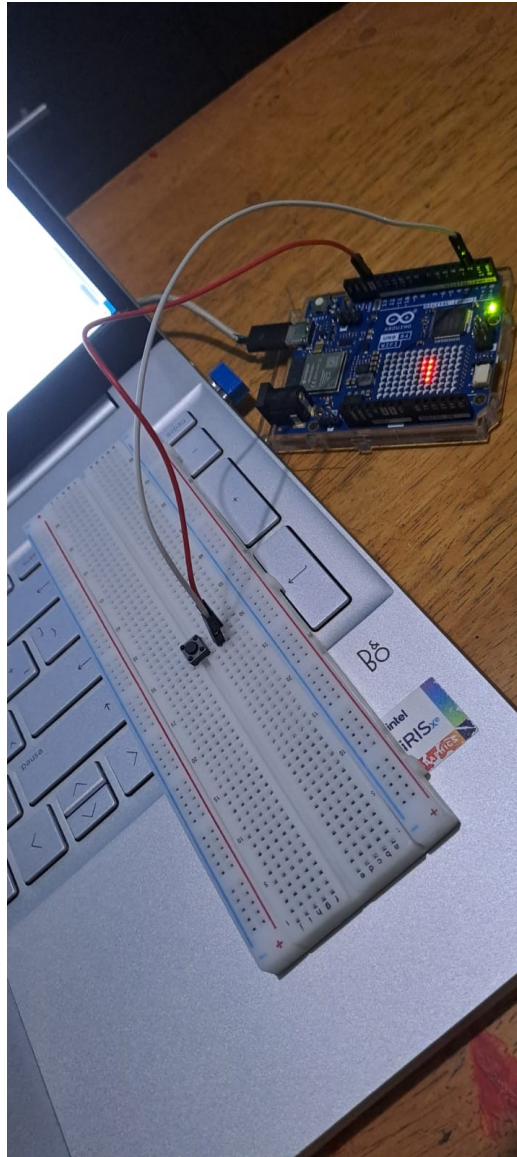


Figura 2. Armado físico: Arduino Uno R4 WiFi con el pulsador conectado y la matriz LED encendida.

El Monitor Serial registró 30 eventos consecutivos, del #1 al #30, con tiempos en microsegundos estrictamente crecientes. La Figura 3 muestra el avance del conteo acumulado y la Figura 4 el tiempo entre cada evento y el anterior; ambas se calcularon únicamente con esos 30 tiempos. La Tabla IV resume los indicadores calculados.

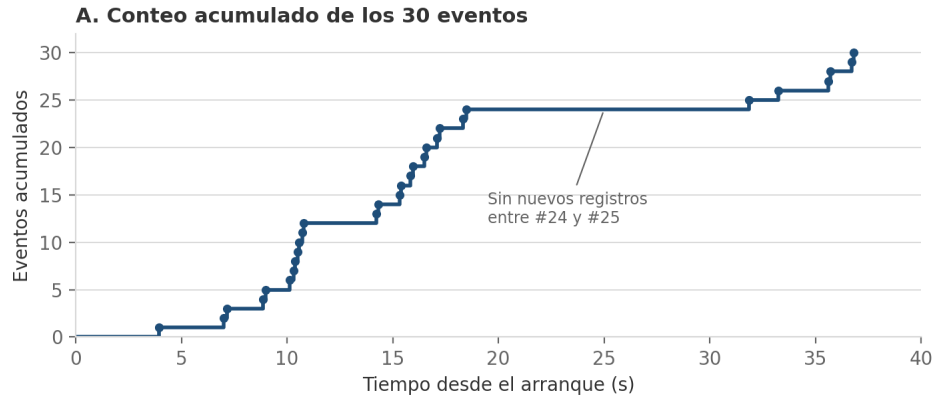


Figura 3. Conteo acumulado de los 30 eventos registrados.

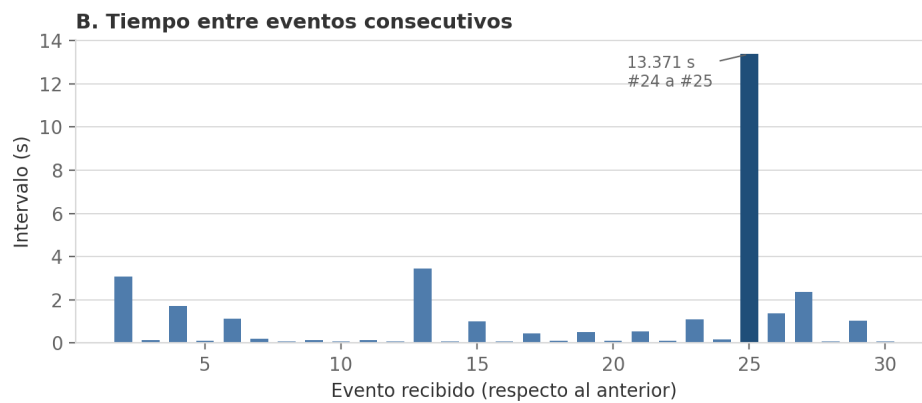


Figura 4. Tiempo entre cada evento y el anterior; el intervalo más largo (13.371 s) ocurrió entre los eventos #24 y #25.

Tabla IV. Indicadores calculados a partir de los 30 eventos registrados.

Indicador calculado	Resultado
Eventos registrados	30 (del #1 al #30)
Lapso entre el primer y el último evento	32.839437 s
Menor intervalo entre eventos	67.595 ms, entre #11 y #12
Mayor intervalo entre eventos	13.370827 s, entre #24 y #25
Mediana de los 29 intervalos	158.913 ms

Las gráficas completas, la tabla de los 30 eventos y las observaciones detalladas se encuentran en Resultados/Resultados_buffercircular.pdf.

VII. Análisis de resultados

Orden e integridad de los eventos. Los 30 tiempos registrados son estrictamente crecientes, sin repeticiones ni saltos hacia atrás, lo que confirma que el buffer circular respetó el orden en que ocurrieron las pulsaciones y que ninguna se perdió durante la prueba.

Efecto del antirrebote. El intervalo más corto observado entre dos eventos fue de 67.595 ms, muy por encima del umbral de 40 ms del filtro. No se observó ningún par de eventos separado por menos de ese umbral, lo que es consistente con que el antirrebote funcionó dentro de la interrupción; no se cuenta, sin embargo, con una medición instrumentada de los rebotes reales del pulsador para cuantificar su eficacia de forma directa.

Capacidad del buffer y ausencia de rechazos. No apareció ningún mensaje de eventos rechazados por buffer lleno durante la prueba, lo que indica que el loop() vació el buffer con suficiente rapidez frente a las 30 pulsaciones registradas, muy por debajo de las 32 posiciones disponibles. La prueba no llegó a documentar la vuelta completa de los índices al inicio del arreglo ni una condición de saturación; ambas situaciones están contempladas en el código, pero requerirían una prueba aparte con más de 32 pulsaciones sin que el loop() las procese.

Independencia entre la interrupción y la animación. La matriz LED se mantuvo en movimiento durante toda la prueba, incluso mientras se presionaba el pulsador repetidamente. Esto confirma que la interrupción, al limitarse a anotar el evento y salir de inmediato, no bloqueó de forma perceptible el resto del programa.

VIII. Conclusiones

Conclusión general. Se cumplió el objetivo de registrar eventos impredecibles sin perder ninguno, combinando una interrupción externa con un buffer circular de tamaño fijo. La interrupción se limitó a anotar el instante de cada pulsación y salir de inmediato, mientras que el loop() procesó esos datos a su propio ritmo y mantuvo la animación de la matriz LED en movimiento constante, sin usar delay() en ningún punto del programa. Las pruebas confirmaron el orden y la integridad de los 30 eventos registrados, aunque quedó pendiente demostrar la vuelta completa del buffer y su respuesta al llenarse.

Conclusión individual (L231000103). Esta práctica me ayudó a entender por qué una interrupción debe ser lo más corta posible: cualquier cosa que tarde dentro de ella —imprimir, calcular, esperar— detiene el resto del programa mientras se ejecuta. Lo que más trabajo me costó fue separar bien las dos velocidades del sistema: el pulsador avisa en cualquier instante impredecible, y el loop() revisa a su propio paso constante, sin que ninguno tenga que esperar al otro gracias al buffer circular. También noté que distinguir el buffer lleno del vacío no es trivial cuando ambos índices pueden coincidir, y que la variable eventosPendientes es la que realmente resuelve esa ambigüedad. Como mejora, agregaría una prueba con más de 32 pulsaciones rápidas para comprobar en la práctica la vuelta completa del buffer y el manejo de la saturación.

Referencias

- | | | | | | | |
|-----|----------|----------------------|------------------------|------------|------------|---|
| [1] | Arduino, | «attachInterrupt()», | Language | Reference. | Disponible | en: |
| | | | | | | https://www.arduino.cc/reference/en/language/functions/external-interrupts/attachinterrupt/ |
| [2] | Arduino, | «digitalRead()», | Language | Reference. | Disponible | en: |
| | | | | | | https://www.arduino.cc/reference/en/language/functions/digital-io/digitalread/ |
| [3] | Arduino, | «micros()», | Language | Reference. | Disponible | en: |
| | | | | | | https://www.arduino.cc/reference/en/language/functions/time/micros/ |
| [4] | Arduino, | «Debounce», | ejemplo | oficial. | Disponible | en: |
| | | | | | | https://docs.arduino.cc/built-in-examples/digital/Debounce/ |
| [5] | Arduino, | «UNO R4 WiFi», | documentación oficial. | | Disponible | en: |
| | | | | | | https://docs.arduino.cc/hardware/uno-r4-wifi |